

Etapas do Pipeline (2–10)

Este guia descreve, com base no código-fonte atual em `src/`, como cada etapa do pipeline é implementada — da etapa **2 (preprocess)** até a etapa **10 (report)**.

A numeração segue o registro central em `src/pipelines/workflow.py` (dicionário `STAGES`), que é a única fonte de verdade sobre quais etapas existem e em que ordem rodam:

#	Nome (<code>--stage</code>)	Classe	Módulo
1	<code>collect</code>	<code>CollectionStage</code>	fora do DAG do DVC — exige aprovação ética, roda manualmente via <code>make collect</code>
2	<code>preprocess</code>	<code>PreprocessingStage</code>	<code>src/pipelines/preprocessing.py</code>
3	<code>label</code>	<code>LabelingStage</code>	<code>src/pipelines/labeling.py</code>
4	<code>psych</code>	<code>PsychologicalStage</code>	<code>src/pipelines/psychological.py</code>
5	<code>embed</code>	<code>EmbeddingStage</code>	<code>src/pipelines/embedding.py</code>
6	<code>features</code>	<code>FeaturesStage</code>	<code>src/pipelines/features.py</code>
7	<code>split</code>	<code>SplittingStage</code>	<code>src/pipelines/splitting.py</code>
8	<code>train</code>	<code>TrainingStage</code>	<code>src/pipelines/training.py</code>
9	<code>evaluate</code>	<code>EvaluationStage</code>	<code>src/pipelines/evaluation.py</code>
10	<code>report</code>	<code>ReportingStage</code>	<code>src/pipelines/reporting.py</code>

Todas as etapas herdam de `PipelineStage` (`src/pipelines/base.py`), que define o contrato comum: `name`, `description`, `required_inputs` (verificados em `check_dependencies` antes de qualquer processamento) e `run(context)`. O orquestrador (`src/main.py` → `pipelines/workflow.run_pipeline`) executa as etapas 2–10, em sequência, quando chamado com `--stage all` (a etapa `collect` fica de fora de propósito). Cada etapa também é uma stage independente no DAG reproduzível do DVC (`dvc.yaml`), com `deps/outs/metrics` declarados.

Um padrão recorrente nas etapas 2–6 é o **processamento incremental por usuário**: cada uma compara os usuários já disponíveis na entrada com os já processados na saída (`select_pending_users`), processa só a diferença, e grava o resultado de cada usuário imediatamente (`write_user_partition`). Isso torna reexecuções seguras e baratas, e permite retomar uma execução interrompida sem reprocessar trabalho já feito — importante em etapas caras (LLM, embeddings) que podem levar horas.

Etapa 2 — Preprocessamento

Objetivo

Consolidar o histórico bruto coletado por usuário, aplicar deduplicação e filtros de qualidade, gerar duas versões de texto (uma preservando semântica para modelos Transformer/LLM, outra agressivamente

normalizada para TF-IDF/léxicos) e descartar tweets/usuários que não sustentam o estudo longitudinal.

Entradas / Saídas

- **Classe:** `PreprocessingStage` em `src/pipelines/preprocessing.py`.
- **Entrada obrigatória** (`required_inputs`): `paths.data.user_histories` — o diretório particionado por usuário produzido pela etapa `collect`.
- **Saída:** `data/interim/tweets_clean/`, particionado por usuário — um `.parquet` por `user_id`, gravado via `write_user_partition` (`src/data/writer.py`).
- **Retorno de `run()`:** contagens de tweets/usuários de entrada e saída, `taxa_retencao`, `usuarios_processados_nesta_execucao` e o caminho gravado.

Implementação (passo a passo)

1. **Descoberta do que falta processar** (`PreprocessingStage.run`): `list_collected_users` (`src/data/reader.py`) lista os usuários já presentes em `user_histories` (entrada) e em `tweets_clean` (já processados); `count_partitioned_rows` conta o total de tweets brutos disponíveis; `select_pending_users(available, already_processed, limit)` calcula a diferença de conjuntos, ordena deterministicamente e aplica `--limit-users`.
2. **Processamento por usuário** (`for user_id in track(pending, ...)`), um de cada vez, sem materializar mais de um histórico simultaneamente:
 - `read_user_history(user_histories, user_id)` lê só o `.parquet` daquele usuário, deduplica por `tweet_id` e ordena por `created_at`;
 - se vazio, pula;
 - `run_preprocessing(user_raw, config, allow_empty=True)` (`src/preprocessing/pipeline.py`) executa a etapa completa (detalhada abaixo);
 - `write_user_partition(user_clean, tweets_clean, user_id)` grava o resultado imediatamente — inclusive quando fica vazio (ex.: usuário inteiramente filtrado por ser conta automatizada), preservando o registro de que o usuário já foi tratado.
3. **`run_preprocessing`** (orquestrador central):
 - valida a entrada contra `RawTweetSchema` (`validate_frame`);
 - aplica os passos em ordem deliberada — "barato antes de caro":
 1. `deduplicate` (`src/preprocessing/cleaning.py`) — remove duplicatas por `tweet_id` e por texto-dentro-do-usuário. Texto idêntico *entre* usuários (correntes, letras de música) é preservado como sinal, não ruído, por padrão;
 2. `filter_by_quality` — usa `configs/collection.yaml:filters` (comprimento mínimo/máximo de caracteres, idioma);
 3. `filter_automated_accounts` — remove usuários com média de tweets/dia acima de `collection.filters.max_tweets_per_day` (contas de notícia/bot/divulgação distorceriam rótulo e features temporais);
 4. `apply_text_processing` — cria as colunas `text_normalized` (`normalize_text`) e `text_clean` (`clean_text`);
 5. `filter_after_cleaning` — descarta tweets que ficaram vazios ou com poucos tokens após a limpeza agressiva;

6. **filter_users_by_activity** — reavaliada **por último**, pois os filtros anteriores podem ter derrubado um usuário abaixo do mínimo de `min_tweets_per_user/min_active_days`;
 - com `allow_empty=True` (usado no laço por usuário), um único usuário zerado é esperado, não uma falha; a saída é validada contra `CleanTweetSchema`.
4. **normalize_text / clean_text** (`src/preprocessing/text.py`) — os dois caminhos de texto, intencionalmente distintos:
 - **normalize_text**: normalização Unicode (NFC), remoção de RT/caracteres de controle, redação de PII por placeholders (e-mail → `EMAIL`, URL → `URL`, menção → `@user`, telefone → `TELEPHONE`), desempacotamento de hashtags (`#depressao` → `depressao`), colapso de caracteres repetidos (`muiooooo` → `muioo`, mantendo ênfase). Preserva caixa, pontuação, emoji e negações — é a entrada dos Transformers/LLM;
 - **clean_text**: minúsculas, remoção de emoji/pontuação/acentos (acentos mantidos por padrão em pt-BR), remoção de stopwords exceto uma *whitelist* (pronomes de 1ª pessoa e negações — `eu`, `não`, `nunca`, `sem` — features centrais na literatura de depressão), filtragem por comprimento mínimo de token. É a entrada de TF-IDF/n-grams/léxicos;
 - **contains_pii** — salvaguarda usada antes de enviar texto ao LLM na etapa 4.
5. **Tokenização/lematização** (`src/preprocessing/tokenization.py`, classe `Tokenizer`) — utilitário usado por etapas posteriores (features), não pelo `PreprocessingStage` diretamente. Usa spaCy (`pt_core_news_sm`) com fallback automático para tokenização por regex se o modelo não estiver instalado.
6. **Stopwords**: `configs/lexicons/stopwords_ptbr.txt` (113 termos), via `utils.lexicons.load_stopwords`.

Configuração relevante

- `configs/preprocessing.yaml`: `deduplication` (`by_tweet_id`, `by_text_within_user`, `by_text_global: false`), `normalization` (placeholders de PII, `unpack_hashtags`, `collapse_repeated_chars: 2`), `cleaning` (`lowercase`, `remove_punctuation`, `remove_emojis`, `remove_accents: false`, `stopwords_whitelist`, `min_token_length: 2`), `tokenization` (`backend: spacy`, `spacy_model: pt_core_news_sm`, `lemmatize: true`), `filters` (`min_tokens_per_tweet: 3`, `drop_empty_after_cleaning: true`).
- `configs/collection.yaml`: fonte de `filters.min_chars_per_tweet / max_chars_per_tweet / max_tweets_per_day / require_language` e de `user_history.min_tweets_per_user / min_active_days`, usados pelos filtros acima.
- `configs/lexicons/stopwords_ptbr.txt`: lista de stopwords pt-BR.

Design notável

- Processamento incremental por usuário com retomada via `select_pending_users` + gravação imediata.
- Contrato de dados validado nas duas pontas (`RawTweetSchema` na entrada, `CleanTweetSchema` na saída).
- `allow_empty=True` no laço por usuário evita que um único usuário totalmente filtrado derrube a execução.

- Duas colunas de texto (`text_normalized` vs. `text_clean`) — decisão central de design: preservar semântica para Transformers/LLM vs. reduzir agressivamente para TF-IDF/léxicos.

Etapa 3 — Rotulagem

Objetivo

Produzir dois níveis de rótulo: (1) sentimento e emoções por tweet, via encoders Transformer; (2) classe do usuário (`controle` / `depressao` / `ideacao_suicida`) por **supervisão fraca**, combinando votos de três fontes ponderadas, com consenso mínimo, amostragem para revisão manual e cálculo de concordância (kappa de Cohen).

Entradas / Saídas

- **Classe:** `LabelingStage` em `src/pipelines/labeling.py`.
- **Entrada obrigatória:** `paths.data.tweets_clean` (saída da etapa 2).
- **Saídas:**
 - `data/interim/tweets_labeled/` — particionado por usuário, com colunas de sentimento/emoção por tweet;
 - `paths.data.user_labels` (parquet único) — um rótulo por usuário;
 - amostra estratificada para revisão manual (`consensus.manual_review_file`), se não vazia;
 - `reports/metrics/labeling_quality.json` — distribuição de classes, concordância com revisão manual, `concordancia_media_fontes`, contagem de usuários rotulados.

Implementação (passo a passo)

Nível tweet — `_label_tweets`:

1. Mesmo padrão de retomada da etapa 2: `list_collected_users` compara `tweets_clean` (disponível) com `tweets_labeled` (já processado); `select_pending_users` calcula os pendentes.
2. `_build_labelers(config)` instancia `SentimentLabeler` (se `labeling.sentiment.enabled`) e `EmotionLabeler` (se `labeling.emotion.enabled`).
3. Para cada usuário pendente (`_label_and_write_user`): lê a partição, aplica `sentiment_labeler.label_frame` (valida contra `LabeledTweetSchema`), depois `emotion_labeler.label_frame` dentro de um `try/except` — falha na rotulação de emoções é degradação aceitável (logada como warning, sem interromper a etapa); falha no classificador de sentimento não é (propaga erro).

SentimentLabeler (`src/labeling/sentiment.py`): `transformers.pipeline` sobre `text_normalized`, com `model_name` primário e `fallback_model_name` em cascata; rótulos abaixo de `min_confidence` viram `indefinido` — decisão explícita para não propagar incerteza como certeza na supervisão fraca a jusante. Produz `sentiment`, `sentiment_score` e `sentiment_polarity`. É explicitamente um classificador determinístico, não um LLM generativo — sentimento é constructo auxiliar (feature/triagem), nunca proxy do risco clínico.

EmotionLabeler (`src/labeling/emotion.py`): pipeline multi-rótulo (`pysentimiento/robertuito-emotion-analysis`) sobre `text_normalized`, gera uma coluna `emotion_<nome>` por emoção em `target_emotions`.

Nível usuário — supervisão fraca (após `_label_tweets`):

4. `read_partitioned(tweets_labeled, ...)` concatena todo o acumulado — essa parte **não é decomponível por usuário**, pois depende de estatísticas populacionais (balanceamento, amostragem estratificada).
5. `assign_user_labels (src/labeling/weak_supervision.py)`:
 - `compute_lexical_evidence`: para cada léxico, calcula por usuário a **proporção** de tweets com ≥ 1 termo (`<lexico>_ratio`, não contagem bruta, para não deixar um tweet repetitivo pesar como vários) e `negative_ratio` (fração de tweets com `sentiment == negativo`);
 - `compute_temporal_persistence`: usa léxicos de risco (`death`, `hopelessness`, `loneliness`) para marcar em quantas janelas de `temporal_persistence.window_days` (30 dias) o usuário mostrou sinal; `has_persistence` exige `windows_with_signal >= 2` e `span_days >= 60` — distingue transtorno persistente de reação pontual a um evento;
 - `_collect_candidate_labels`: deriva rótulo candidato do `source_group` mais frequente entre os tweets do usuário. **Nota**: `source_group` só é preenchido nos tweets-semente da busca inicial — os tweets do histórico retrospectivo (maior parte do dataset após a etapa 2) têm `source_group` nulo, então o voto `collection_group` raramente dispara sobre o conjunto rotulado. Isso é consistente com o achado conhecido de que `concordancia_media_fontes` tende a ficar artificialmente próxima de `1.0` (as fontes que efetivamente votam — léxica e temporal — tendem a concordar entre si);
 - `label_from_lexical_evidence`: aplica limiares de `labeling.yaml:user_labeling.lexical_thresholds` — primeiro `ideacao_suicida` (por `death_ratio/death_hits`), depois `depressao` (exige `negative_ratio` mínimo e `hopelessness_ratio` ou `loneliness_ratio`), senão `controle`;
 - `_collect_votes + resolve_consensus`: soma pesos por classe (empates resolvidos por `CLASS_PRECEDENCE: ideacao_suicida > depressao > controle`); concordância abaixo de `consensus.min_agreement` (0.60) $\rightarrow$ indefinido;
 - `_build_user_record`: inclui `user_label_multilabel` (união de todas as classes de risco votadas, preservando coocorrência).
6. `load_manual_labels / apply_manual_labels (src/labeling/validation.py)` — revisão humana sempre vence sobre a supervisão fraca.
7. `compute_agreement` — concordância simples e **kappa de Cohen** entre `user_label` e `manual_label`.
8. `sample_for_manual_review` — amostragem **estratificada por classe** de tamanho `manual_review_sample_size` (200), semente fixa; garante representação mínima de `ideacao_suicida` (tipicamente minoritária).
9. `drop_undecided` — remove usuários `indefinido` se `consensus.drop_undecided: true` (padrão).
10. Validação final (`UserLabelSchema`), `check_class_balance` (compara com `collection.sampling.max_class_imbalance_ratio`), gravação do parquet e de `labeling_quality.json`.

Configuração relevante

- `configs/labeling.yaml`:
 - `sentiment: cardiffnlp/twitter-xlm-roberta-base-sentiment (+ fallback pysentimiento/bertweet-pt-sentiment)`, `min_confidence: 0.50`;
 - `emotion: pysentimiento/robertuito-emotion-analysis`;

- `user_labeling.sources`: pesos `collection_group=0.40`, `lexical_evidence=0.35`, `temporal_persistence=0.25`;
- `user_labeling.lexical_thresholds`, `temporal_persistence`, `consensus` (`min_agreement=0.60`, `drop_undecided=true`, `manual_review_sample_size=200`);
- `class_precedence`: [`ideacao_suicida`, `depressao`, `controle`];
- `control_is_screening_negative_only`: `true` — documenta que `controle` significa "sem sinais detectados na amostra coletada", não ausência clínica confirmada.
- `configs/lexicons/`: `death.txt` (23 termos), `hopelessness.txt` (24), `loneliness.txt` (21), `isolation.txt` (21), `negative_emotion.txt` (44), `insomnia.txt` (23) — carregados via `utils.lexicons.load_lexicons`.

Design notável

- Dois ritmos de execução: rotulação por tweet é incremental/retomável; rotulação por usuário roda uma única vez sobre o acumulado (depende de estatísticas globais).
- Tratamento de erro assimétrico: emoção é recuperável, sentimento não.
- Kappa de Cohen, amostragem estratificada e revisão manual são parte formal do pipeline, com resultado persistido em `labeling_quality.json`.
- **Limitação conhecida**: o voto `collection_group` depende de `source_group`, normalmente nulo nos tweets de histórico — reduz, na prática, a supervisão fraca às fontes léxica e temporal, o que deve ser considerado ao interpretar `concordancia_media_fontes`.

Etapas 4 — Escores Psicológicos

Objetivo

Extrair, para cada usuário, um vetor psicológico de cinco dimensões (`tristeza`, `isolamento`, `esperanca`, `ansiedade`, `risco_suicida`, todas em `[0,1]`) a partir de lotes de tweets, usando um LLM local via Ollama. É a etapa mais lenta do pipeline e a única que depende de um serviço externo ao processo — por isso é opcional e falha de forma controlada (o pipeline segue sem o grupo `psychological`, com aviso explícito).

Entradas / Saídas

- **Classe**: `PsychologicalStage` em `src/pipelines/psychological.py`.
- **Entrada obrigatória**: `paths.data.tweets_clean`. A origem real (`_resolve_source_dir`) prefere `tweets_labeled` se já existir (tem sentimento), senão cai para `tweets_clean`.
- **Saída**: partições por usuário em `paths.data.psychological_scores` (`schemas.tweets.PsychologicalScoreSchema`: uma linha por lote de tweets processado, com `user_id`, `batch_index`, `n_tweets`, `window_start/end`, as 5 dimensões, `model` e `prompt_version`).

Implementação (passo a passo)

1. Se `config.llm.psychological_features.enabled` for `false`, a etapa é pulada.
2. Seleção de usuários pendentes (mesmo padrão de `select_pending_users` das etapas anteriores) — processamento incremental por usuário.
3. `_create_extractor`: instancia `PsychologicalExtractor(config.llm)` e `client.ensure_model(...)` verifica se o modelo está disponível no servidor Ollama; se indisponível (ou pacote `ollama` ausente), a etapa é **pulada com aviso**, sem derrubar o pipeline.

4. `_extract_pending_users`: para cada usuário pendente, lê a partição, chama `extractor.extract_frame`, valida contra `PsychologicalScoreSchema` e grava imediatamente (`write_user_partition`) — a gravação por usuário evita perder horas de inferência já feita em caso de interrupção.
5. `PsychologicalExtractor.extract_frame` (`src/labeling/llm.py`): ordena tweets por `[user_id, created_at]`, particiona por usuário, quebra em lotes de `batch_size_tweets` (20) preservando `batch_index` e `window_start/window_end`; dispara as chamadas ao LLM **em paralelo** com `ThreadPoolExecutor(max_workers=max_concurrency)` (4) — justificado por ser I/O bloqueante de rede ao Ollama, não trabalho de CPU.
6. `extract_batch` (por thread): monta o prompt (`build_psychological_prompt`), chama `client.generate(...)` com `temperature=0.0`, `seed=42`, `num_ctx=8192`; valida a resposta como `PsychologicalVector` (Pydantic, `Field(ge=0, le=1)`); em falha de validação, **repete** até `max_repairs` vezes (2, logo até 3 tentativas); se todas falharem, descarta o lote — nunca "inventa" um valor.
7. `build_psychological_prompt` (`src/labeling/prompt.py`): se `safeguards.require_pii_scrubbed_input` (`true`), verifica com `contains_pii` que o texto já foi higienizado pelo preprocessing; calcula orçamento de caracteres (`max_prompt_chars=24000`) e trunca pelo fim se necessário; retorna um `Prompt` com `version` (`1.0.0`) gravada em cada score para rastreabilidade.
8. `OllamaClient.generate`: cache por hash do payload (`{system, user, version, model, temperature, seed}`) em `data/interim/llm_cache` — só a primeira tentativa usa cache, reparos forçam nova chamada; retry com backoff exponencial (`retry_attempts=3`, `backoff_seconds=5`, dobrando a cada falha); `extract_json_object` extrai o JSON da resposta, tolerante a texto explicativo ao redor.
9. Consolidação final: relista `processed_users`; se vazio, retorna skip; senão retorna `n_lotes`, `n_usuarios`, `usuarios_processados_nesta_execucao`, `modelo` e `versao_prompt`.

Configuração relevante (`configs/llm.yaml`)

- `ollama`: `host`, `timeout_seconds=120`, `keep_alive=5m`, `auto_pull=false` (falha explícita em vez de baixar GBs silenciosamente), `retry_attempts=3`, `backoff_seconds=5`.
- `psychological_features`: `enabled=true`, `model=llama3.2`, `temperature=0.0`, `seed=42`, `num_ctx=8192`, `max_repairs=2`, `batch_size_tweets=20`, `max_concurrency=4` (dimensionado para GPU V100S-32GB), `cache.enabled=true`.
- `prompts.version="1.0.0"`, templates pt-BR com regras (JSON apenas, `esperanca` é dimensão positiva, não inferir diagnóstico clínico, usar 0.5 se conteúdo insuficiente).
- `safeguards`: `require_pii_scrubbed_input=true`, `max_prompt_chars=24000`, `log_prompt_content=false` (nunca loga conteúdo do prompt, só hash/metadados — LGPD arts. 11 e 33; processamento 100% local via Ollama pela mesma razão).

Design notável

- Processamento incremental por usuário com retomada e gravação imediata.
- Cache por hash de prompt: reexecução não reprocessa o mesmo prompt/modelo/parâmetros.
- Saída validada com reparo limitado (Pydantic + `max_repairs`), preferindo descartar a inventar valor.
- Concorrência via threads (I/O-bound), não processos.

- Falha graciosa: indisponibilidade do LLM não derruba o pipeline, apenas o grupo `psychological` fica ausente da matriz de features (mensurável pelo Ablation Study).
- Privacidade por design: execução local, PII removida antes do envio, prompt nunca logado.

Etapa 5 — Embeddings Semânticos

Objetivo

Gerar vetores densos por tweet usando um encoder Transformer, e persistir esses vetores brutos em disco, separando essa etapa (cara em GPU) da agregação em nível de usuário — os vetores podem ser reaproveitados em todas as agregações e modelos subsequentes sem recodificar milhões de textos a cada experimento.

Entradas / Saídas

- **Classe:** `EmbeddingStage` em `src/pipelines/embedding.py`.
- **Entrada obrigatória:** `paths.data.tweets_clean` (origem real prefere `tweets_labeled` se disponível).
- **Saída final por modelo/encoder:** `<embeddings_dir>/<nome>.npz` (matriz (`n_tweets`, `dim`)) + `<embeddings_dir>/<nome>_index.parquet` (coluna `user_id` por linha, mesma ordem), via `save_embeddings`.
- **Cache intermediário por usuário:** `<embeddings_dir>/_cache/<nome>/<user_id>.npz` (+ arquivo `.owner`).

Implementação (passo a passo)

1. Se `config.features.semantic.enabled` for `false`, a etapa é pulada.
2. `_resolve_requested_models`: por padrão só o `primary_model` (`neuralmind/bert-base-portuguese-cased`) roda; com `--all-encoders`, adiciona os demais `config.models` (`bertimbau`, `roberta`, `gemma`, `llama`).
3. Para cada encoder (`_process_encoder`): lista usuários já codificados no cache, calcula pendentes via `select_pending_users` — processamento incremental por usuário, igual à etapa 4 — codifica os pendentes (`_encode_pending_users`) e consolida o cache (`_consolidate_cache`).
4. `_encode_pending_users`: instancia `EmbeddingEncoder(model_name, config)` (falha isolada por encoder, sem derrubar o pipeline); para cada usuário pendente, `_encode_and_cache_user` lê a partição, chama `encoder.encode`, grava `.npz` no cache e um arquivo `.owner` com o `user_id` pseudonimizado real (o nome do arquivo é a chave de partição bruta da coleta, que pode diferir do pseudônimo usado no resto do pipeline).
5. `EmbeddingEncoder.encode` (`src/features/semantic.py`): carrega `AutoTokenizer/AutoModel` sob demanda (senão `MissingDependencyError`); move para `self.device` (`resolve_device("auto")`); `model.eval()` e `torch.set_grad_enabled(False)` (só *forward pass*); processa em lotes de `batch_size` (128, dimensionado para GPU V100S-32GB); tokeniza com `padding`, `truncation`, `max_length=128`; **pooling** `mean` (padrão, respeita a máscara de atenção) ou `cls`; se `normalize` (`true`), aplica normalização L2 (necessária para similaridade por cosseno).
6. `_consolidate_cache`: empilha todos os `.npz` cacheados, resolve o `user_id` real via `.owner` (ou relê a partição de origem, gravando o `.owner` para consolidações futuras), expande a lista de owners por linha de embedding, chama `save_embeddings`.

7. Retorno: `n_tweets_codificados`, `modelos`, `dimensoes` por modelo, `written`. Se nenhum encoder produziu saída, retorna skip.

Funções auxiliares no mesmo módulo, usadas pela **etapa 6** (não pela etapa 5 em si): `aggregate_embeddings` (agrega por usuário conforme `user_aggregations: [mean, std]` — média captura o "centro semântico" do perfil, desvio captura a dispersão temática) e `build_semantic_features` (codifica e agrega em um só passo, para comparações pontuais). A redução dimensional (PCA, `n_components=64`, ajustado só no split de treino) é aplicada depois, dentro do pipeline de features/modelagem.

Configuração relevante (`configs/features.yaml`, seção `semantic`)

- `enabled: true, primary_model: neuralmind/bert-base-portuguese-cased`.
- `models: bertimbau, roberta (pucpr/biobertpt-all), gemma (embeddinggemma), llama (llama3.2)` — só com `--all-encoders`.
- `pooling: mean, max_length: 128, batch_size: 128, device: auto, normalize: true`.
- `user_aggregations: [mean, std]`.
- `reduction: {method: pca, n_components: 64, random_state: 42}`.
- `groups.semantic: true` — chave do Ablation Study em `evaluation.yaml`.

Design notável

- Cache em dois níveis: por usuário (retomável) e consolidado por modelo.
- Resolução de identidade (`owner`) desacoplada da chave de cache.
- Múltiplos encoders opcionais: primário sempre roda, os demais só sob demanda explícita.
- Falha graciosa por encoder.
- GPU/device automático e lotes maiores (não há gradientes em memória).
- Mean pooling com máscara de atenção + normalização L2, para tornar os vetores comparáveis por similaridade de cosseno.

Etapa 6 — Engenharia de Features

Objetivo

Transformar os tweets rotulados em **uma linha de atributos por usuário** (a unidade amostral do projeto), reunindo seis grupos de features (linguísticas, emocionais, semânticas, temporais, comportamentais e psicológicas) mais colunas descritivas de perfil, com imputação de ausentes e junção com o rótulo do usuário.

Entradas / Saídas

- **Classe:** `FeaturesStage` em `src/pipelines/features.py`.
- **Entradas obrigatórias:** `paths.data.tweets_labeled` e `paths.data.user_labels`.
- **Entradas opcionais:** `paths.data.user_metadata` (features de audiência) e `paths.data.psychological_scores` (etapa 4) — se ausentes, os grupos correspondentes são apenas logados como omitidos.
- **Entrada adicional para o grupo semântico:** `embeddings.npy` / `_index.parquet` de `paths.data.embeddings` (etapa 5).
- **Saída principal:** `paths.data.user_features` (parquet, uma linha por usuário).

- **Saídas auxiliares:** `paths.data.user_features_raw` (checkpoint particionado por usuário, retomável), `reports/metrics/features_summary.json` e o manifesto do dataset.

Implementação (passo a passo)

1. **Processamento incremental por usuário** (`_build_pending_user_features` → `_build_and_write_user_row`): calcula pendentes, lê tweets/metadados/scores psicológicos por usuário, chama `build_user_features_raw` e grava imediatamente.
2. **`build_user_features_raw`** (`src/features/builder.py`), a metade *decomponível por usuário*:
 - `build_profile_columns`: `n_tweets`, `active_days`, `span_days`, `first_tweet_at`, `last_tweet_at`;
 - une, condicionalmente por grupo ativo (`config.features.groups`): `_join_linguistic_group` (`features/linguistic.py`: razões de léxico, comprimento de texto, diversidade lexical — TTR + MTLD, uso de pronomes, n-gramas TF-IDF via `features/ngrams.py`), `_join_emotional_group` (`features/emotional.py`: distribuição de sentimento, confiança, intensidade de emoções), `_join_temporal_group` (`features/temporal.py`: volume, atividade noturna, entropia circadiana, tendência de sentimento, intensificação de risco), `_join_behavioral_group` (`features/behavioral.py`: engajamento, audiência, razões seguidor/seguindo e de respostas), `_join_semantic_group` (agregação dos embeddings da etapa 5) e `_join_psychological_group` (vetor da etapa 4);
 - **remove as colunas que vazam o rótulo** via `_drop_label_leaking_columns` (ver seção dedicada abaixo);
 - promove `temp_night_activity_ratio` a coluna de perfil (usada nas fatias de avaliação da etapa 9);
 - filtra usuários abaixo de `aggregation.min_tweets_per_user`.
3. **Finalização única, sobre a população inteira** (`_finalize_features` → `features.builder.finalize_user_features`), roda uma vez após todos os pendentes serem processados:
 - concatena o acumulado de `user_features_raw` e remove duplicatas de `user_id`;
 - trata ausentes (`handle_missing_values`): normaliza NaN para null, cria indicadores binários `<coluna>_is_missing`, imputa conforme `aggregation.missing_strategy` (median — mediana **global**, com `fallback 0.0` —, zero ou `keep_nan`);
 - une o rótulo do usuário (`inner join` com `user_labels`);
 - agrega e une os embeddings semânticos (`aggregate_embeddings`, `mean/std`, `left join + fill_null(0.0)`).
4. **Validação e persistência:** `schemas.features.validate_feature_matrix(features, allow_nan=False)` garante `user_id` presente e único (crítico para não vazarem o mesmo usuário entre treino e teste na etapa 7), ao menos uma coluna de atributo e ausência de valores não finitos; grava o parquet final e `features_summary.json` (hash da matriz, contagem por grupo, resumo de ausentes).

Prevenção de vazamento de rótulo

Commit `fix(features)`: **prevenir vazamento de rótulos na matriz de recursos**, endereçando um vazamento direto entre a regra de rotulação por supervisão fraca

(`labeling/weak_supervision.py::label_from_lexical_evidence`) e as features do classificador:

- em `configs/features.yaml`, os léxicos `death`, `loneliness` e `hopelessness` de `linguistic.lexicon_ratios` foram comentados (mantidos apenas `isolation`, `negative_emotion`, `insomnia`), pois são exatamente os léxicos usados como limiar para atribuir o rótulo;
- em `src/features/builder.py`, a constante `LABEL_LEAKING_COLUMNS = ("emo_negativo_ratio", "emo_negativo_positive_ratio")` e a função `_drop_label_leaking_columns` (chamada em `build_user_features_raw` logo após a junção de todos os grupos) removem essas duas colunas — derivadas do mesmo `negative_ratio` usado como limiar da classe `depressao`, então vazariam a regra mesmo sem vir de um léxico explicitamente listado — e logam um `warning` com o que foi removido;
- testes dedicados em `tests/test_features.py` cobrem remoção e idempotência quando as colunas não existem.

A imputação por mediana em `handle_missing_values` é documentada como um vazamento leve e aceito (estatística sobre a população inteira, não só o treino) — a alternativa rigorosa (mover para dentro do `Pipeline` do scikit-learn) está registrada em `docs/guides/architecture.md`.

Configuração relevante (`configs/features.yaml`)

- `groups`: liga/desliga cada um dos 6 grupos — unidade do Ablation Study (H2–H4).
- `linguistic.lexicon_ratios`, `ngrams` (TF-IDF, `ngram_range [1,2]`, `max_features 3000`, `min_df 5`, `max_df 0.85`).
- `emotional.aggregations` (`mean`, `std`, `max`, `p90`).
- `semantic.primary_model`, `pooling`, `user_aggregations`, `reduction`.
- `temporal.insomnia_window`, `min_days_for_trend`, `recent_window_days`.
- `behavioral.log_transform`, `aggregations`.
- `psychological.dimensions`, `granularity`, `batch_size_tweets`.
- `aggregation.min_tweets_per_user` (30), `add_missing_indicators`, `missing_strategy`.
- `scaling` (`standard/robust/none`, `fit_on_train_only: true` — o escalonamento real é aplicado dentro do `Pipeline` de treino, não nesta etapa).

Etapas 7 — Divisão dos Dados

Objetivo

Atribuir cada usuário (nunca cada tweet) a uma partição — treino / validação / teste — e a um fold de validação cruzada, garantindo estratificação por classe e **zero vazamento de grupo** (nenhum usuário aparece em mais de uma partição).

Entradas / Saídas

- **Classe:** `SplittingStage` em `src/pipelines/splitting.py`.
- **Entrada obrigatória:** `paths.data.user_features` (colunas `user_id` e `user_label`).
- **Saída:** `paths.data.splits` (parquet com `user_id`, `user_label`, `split`, `fold`).
- Lógica delegada a `src/data/splitter.py`.

Implementação (passo a passo)

1. `SplittingStage.run` lê `user_id/user_label` e chama `build_split_table(features, config.general.split, config.general.cross_validation, random_seed)`, que encadeia `create_splits + assign_folds`.
2. **`create_splits`**: valida que a classe menos frequente tem ao menos 3 usuários (senão `InsufficientDataError`). Usa `sklearn.train_test_split` **duas vezes**, sempre com `stratify=` sobre `user_label`:
 1. separa `test` do restante (`test_size=0.20`);
 2. separa `val` do restante (recalculando a fração relativa para que `val_size=0.10` continue sendo fração do **total**).

Chama `check_no_group_leakage` três vezes (`train×test`, `val×test`, `train×val`).

3. **`assign_folds`**: filtra apenas usuários de desenvolvimento (`split != "test"`) — o teste nunca participa da CV e recebe `fold = -1`. Usa `StratifiedKFold(n_splits=5, shuffle=True, random_state=seed)` sobre os usuários de desenvolvimento.
4. Validação da tabela final contra `schemas.users.SplitSchema` (pandera, `strict=True`): `user_id` único e casado com `PSEUDONYM_REGEX`, `user_label` em `VALID_LABELS`, `split` em `{"train", "val", "test"}`, `fold >= -1`.
5. **Reafirmação da verificação de vazamento**: `SplittingStage.run` chama `check_no_group_leakage(train_users, test_users)` uma segunda vez, de forma independente da checagem já feita dentro de `create_splits` — defesa em profundidade sobre a garantia central do projeto.
6. Grava o parquet e retorna estatísticas: distribuição por partição, distribuição cruzada `split × user_label`, número de folds.

Configuração relevante (`configs/config.yaml`)

```
split:
  group_column: user_id
  stratify_column: user_label
  test_size: 0.20
  val_size: 0.10
  shuffle: true

cross_validation:
  n_splits: 5
  n_repeats: 1
  strategy: stratified_group_kfold
```

mais `project.random_seed: 42`, usado tanto no split quanto nos folds.

Design notável

- Unidade amostral = usuário: a mesma pessoa nunca fica dividida entre partições (evitaria que o modelo aprendesse a reconhecer o autor pelo estilo em vez do sinal clínico).

- Estratificação dupla (split e folds), importante para a classe minoritária `ideacao_suicida`.
- Vazamento de grupo verificado em quatro pontos (três dentro de `create_splits`, mais uma quarta redundante em `SplittingStage.run`).
- Teste tocado uma única vez (`fold = -1`).

Etapa 8 — Treinamento

Objetivo

Executar a validação cruzada (estimativa de incerteza e insumo para os testes estatísticos da etapa 9) e treinar a versão final de cada modelo selecionado sobre o split `train`, persistindo os artefatos.

Entradas / Saídas

- **Classe:** `TrainingStage` em `src/pipelines/training.py`.
- **Entradas obrigatórias:** `data.user_features` e `data.splits`.
- **Entradas opcionais:** tweets rotulados (`data.tweets_labeled`, via `load_user_texts`) para Transformer/LLM; sequências de embeddings por tweet (`data.embeddings`, via `load_user_sequences`) para a BiLSTM.
- **Saídas:** modelos treinados (`.joblib + _metadata.json`) em `paths.models.artifacts`; `reports/metrics/cross_validation.json` (resumo por modelo); `reports/metrics/cv_fold_scores.json` (scores por fold, usados na etapa 9 para Wilcoxon/Friedman).

Implementação (passo a passo)

1. **Carregamento:** `development` (train + val, split $\neq$ "test") alimenta a validação cruzada; `train` alimenta o treino final. O split `test` nunca é tocado nesta etapa.
2. **Textos/sequências:** tweets agrupados por usuário em ordem cronológica; embeddings por tweet reagrupados por usuário, usando o encoder primário (`config.features.semantic.primary_model`).
3. **Instanciação dos modelos** (`create_models`, Factory Pattern em `src/models/factory.py`): a partir de `config.models.select(...)` (`model_params.yaml`), respeitando `--include-exploratory` e `--models`.
4. **Validação cruzada** (`src/training/cross_validation.py`, pulável com `--skip-cv`):
 - `cross_validate_all` itera os specs de modelo e chama `cross_validate_model`;
 - os **folds vêm da tabela de partições** (coluna `fold`), não são recalculados — garante que todos os modelos vejam os mesmos blocos, condição necessária para os testes pareados da etapa 9;
 - em cada fold, um modelo é recriado do zero, treinado e avaliado (`compute_metrics`), retornando a métrica principal (`evaluation.metrics.primary, f1_macro`);
 - `_summarize_fold_scores` calcula média, desvio-padrão e IC 95% pela **distribuição t** (poucos folds — a aproximação normal subestimaria a largura);
 - `n_splits = config.general.cross_validation.n_splits` (5);
 - scores por fold gravados em `cv_fold_scores.json`, restritos aos modelos que completaram o **mesmo número de folds**.
5. **Treinamento final** (`src/training/trainer.py`): para cada modelo, resolve `feature_groups`, monta o `UserDataset` de treino, verifica disponibilidade de entradas (texto/sequências) e treina + persiste (`train_model` $\rightarrow$ `model.fit(dataset)`) dentro de `log_duration`). Persistência via

`models.persistence.save_model`: `.joblib` (`joblib.dump`) + JSON de metadados (versão do projeto, SHA do git, ambiente, hash do dataset via `hash_dataframe(...)[:16]`) — rastreia código + ambiente + dados. Se `context.tracker` (MLflow) estiver configurado, cada modelo roda dentro de `tracker.run(...)`. Falhas de treino de um modelo são isoladas — não derrubam a comparação inteira.

Modelos disponíveis (`src/models/`)

- `base.py` — `BaseUserClassifier` (ABC: `fit`, `predict_proba`, `predict`, `check_fitted`, `validate_dataset`, `describe`) e `UserDataset` (features tabulares + textos + sequências, alinhados por `user_id`).
- `factory.py` — `create_model/create_models`; `TABULAR_ESTIMATORS` (`dummy`, `logistic_regression`, `random_forest`, `xgboost`, `lightgbm`), `SEQUENCE_ESTIMATORS` (`bilstm`, `lstm`, `cnn_text`), mais `transformer`, `llm`, `hybrid`.
- `traditional.py` — `TabularClassifier`: Pipeline scikit-learn (scaler opcional + estimador); escalonamento pulado para modelos de árvore.
- `deep.py` — `SequenceClassifier` (BiLSTM/LSTM): rede recorrente PyTorch sobre a **sequência cronológica de embeddings de tweets** por usuário, com pooling mascarado, `CrossEntropyLoss` ponderada por classe, parada antecipada.
- `transformer.py` — `TransformerClassifier` (BERTimbau/RobERTa/DeBERTa): fine-tuning hierárquico em duas etapas (nível tweet, depois agregação de probabilidades por usuário — `mean/max/majority`).
- `llm.py` — `LLMClassifier`: classificação via Ollama, zero-shot ou few-shot; não é treinado (`fit` só seleciona exemplos few-shot).
- `hybrid.py` — `HybridClassifier` (`hybrid_xgboost`, contribuição metodológica principal): `ColumnTransformer` com PCA+StandardScaler no bloco semântico e StandardScaler nos atributos estruturados, alimentando um XGBoost — equilibra o desequilíbrio dimensional entre embeddings e atributos estruturados.

Configuração relevante (`configs/model_params.yaml`)

- `baseline`: `dummy` (`strategy: stratified`) e `tfidf_logistic` (só `feature_groups: [linguistic]`) — sempre executados, piso de comparação.
- `traditional`: `xgboost` (escopo `comparison`) + exploratórios.
- `deep`: `bilstm` (escopo `comparison`, `epochs=30`, `patience=5`) + exploratórios `lstm`, `cnn_text`.
- `transformer`: `bertimbau` (escopo `comparison`, `fp16: true`) + exploratórios `roberta`, `deberta`.
- `llm`: `ollama_primary` (`llama3.2`, `mode: few_shot`) + exploratórios.
- `hybrid`: `hybrid_xgboost` (escopo `comparison`), `feature_groups: [semantic, emotional, temporal, behavioral, psychological, linguistic]`.
- `class_imbalance.strategy: class_weight` (SMOTE deliberadamente não usado sobre embeddings).

Etapa 9 — Avaliação

Objetivo

Avaliar todos os modelos treinados no split `test` (tocado uma única vez), gerar comparações estatísticas entre modelos, rodar o Ablation Study e a análise de interpretabilidade, e gravar os relatórios finais. Qualquer ajuste

feito após observar o resultado no teste invalidaria a estimativa de generalização.

Entradas / Saídas

- **Classe:** `EvaluationStage` em `src/pipelines/evaluation.py`.
- **Entradas obrigatórias:** `data.user_features`, `data.splits`, `models.artifacts` (etapa 8).
- **Entradas opcionais:** textos/sequências (mesma lógica da etapa 8); `reports/metrics/cv_fold_scores.json` (necessário para Wilcoxon/Friedman).
- **Saídas:** `reports/metrics/evaluation.json`, `reports/tables/model_comparison.csv`, `reports/evaluation_report.md`, `reports/metrics/predictions.csv`, `reports/ablation/ablation.json + ablation_summary.csv`, `reports/interpretability/*.csv` (permutação, importância por grupo, SHAP).

Implementação (passo a passo)

1. Lê `features/splits`, separa `train` (usado só no Ablation Study) e `test`. Se `test` estiver vazio, a etapa é pulada.
2. **Avaliação por modelo** (`_evaluate_models`): itera os `.joblib` em `models.artifacts`, monta o `UserDataSet` de teste restrito aos `feature_groups` do spec do modelo, chama `Evaluator(config).evaluate(model, dataset, profile=test)`. Falhas por modelo são isoladas.
3. **Evaluator.evaluate** (`src/evaluation/evaluator.py`), ponto único de avaliação para todas as famílias de modelo:
 - `predict_proba` → `argmax`; `compute_metrics` (accuracy, precision/recall/F1 macro e weighted, ROC-AUC OvR, PR-AUC macro, MCC);
 - `compute_per_class_metrics`, `compute_confusion_matrix`;
 - **Incerteza:** `bootstrap_confidence_interval` sobre a métrica principal (`n_bootstrap=1000`, IC percentílico 95%);
 - **Calibração** (se habilitada): Brier score + ECE/MCE por bins (`src/evaluation/calibration.py`);
 - **Avaliação por fatias** (se habilitada): `evaluate_all_slices` (`src/evaluation/slices.py`) — fatias **comportamentais** (volume de tweets, janela de observação, atividade noturna), não demográficas, por minimização de dados/LGPD; alerta quando a lacuna entre a melhor e a pior fatia excede `max_acceptable_gap`;
 - `Evaluator.compare` monta a tabela comparativa ordenada pela métrica principal.
4. **Testes de significância estatística** (`src/evaluation/statistics.py`):
 - **Wilcoxon pareado + Friedman/Nemenyi:** usa os scores por fold da etapa 8; com ≥ 3 modelos, roda Friedman (ranking médio) + diferença crítica de Nemenyi; Wilcoxon par a par entre todos os pares, com correção de Holm-Bonferroni e tamanho de efeito (delta de Cliff);
 - **McNemar** (feature do commit "adicionar relatório de comparação par a par de McNemar"): compara cada modelo contra o **melhor modelo**, no mesmo conjunto de teste; usa teste binomial exato se discordâncias < 25 , senão qui-quadrado com correção de continuidade de Yates.
5. **Ablation Study** (`src/evaluation/ablation.py`, pulável com `--skip-ablation`): testa as hipóteses H2/H3/H4, removendo grupos de atributos do modelo `hybrid_xgboost`. Dois modos: **leave-one-out** (contribuição marginal) e **only-one** (contribuição absoluta), cada configuração repetida `n_repeats=5` vezes com sementes diferentes. Resultado inclui `baseline`, `leave_one_out`, `only_one` e um `ranking` de grupos por contribuição marginal.

6. **Interpretabilidade** (`src/interpretability/`): seleciona, entre os modelos com `pipeline_` scikit-learn persistido, o de melhor métrica principal. **Importância por permutação** (`sklearn.inspection.permutation_importance`, `n_repeats=10`, `scoring=f1_macro`, calculada no teste), agregada por grupo — complementa o Ablation Study por caminho independente (ablação mede impacto de *remove*; importância mede o quanto o modelo já treinado *usa*). **SHAP** (`TreeExplainer/LinearExplainer/KernelExplainer` conforme config); para o modelo híbrido, explica o espaço **transformado** (pós-PCA), renomeando features para evitar leitura equivocada de componentes de PCA como atributos originais.
7. **Geração de relatórios** (`src/evaluation/reports.py`): JSON, CSV e Markdown (`evaluation_report.md`) com seções de comparação de modelos (com IC), melhor modelo, desempenho detalhado por modelo, testes estatísticos (Friedman, Wilcoxon+Holm, McNemar), Ablation Study e uma seção fixa de **Limitações** (viés de seleção do rótulo `controle`, supervisão fraca, ausência de atributos demográficos, natureza de triagem para pesquisa — não diagnóstico clínico).

Configuração relevante (`configs/evaluation.yaml`)

- `metrics.primary`: `f1_macro`; `highlight` destaca `recall_ideacao_suicida` e `pr_auc_ideacao_suicida` (maior custo clínico de falso negativo).
- `uncertainty`: `bootstrap`, `n_bootstrap=1000`, `confidence_level=0.95`.
- `calibration`: `n_bins=10`, Brier score + ECE.
- `statistics`: `mcnemar.enabled=true`, `wilcoxon.enabled=true`, `friedman.enabled=true`, `alpha=0.05`, `multiple_comparison_correction`: `holm`, `effect_size`: `cliffs_delta`.
- `slices`: `volume_tweets`, `janela_observacao`, `atividade_noturna`; `min_samples_per_slice=20`; `max_acceptable_gap=0.15`.
- `ablation`: `base_model`: `hybrid_xgboost`, `grupos` [`linguistic`, `emotional`, `semantic`, `temporal`, `behavioral`, `psychological`], `mode`: `leave_one_out`, `include_only_one=true`, `n_repeats=5`.
- `interpretability.shap`: `explainer`: `tree`, `max_display=25`, `sample_size=500`; `permutation_importance`: `n_repeats=10`, `scoring`: `f1_macro`.
- `regression_thresholds`: F1 macro mínimo para `hybrid_xgboost` (0.70) e `xgboost` (0.65) — usados como *gate* de CI.
- `reporting`: `formats`: [`json`, `csv`, `md`], `save_predictions`: `true`, `generate_model_card`: `true`.

Etapas 10 — Relatório Final

Objetivo

Fechar o pipeline gerando todo o material de comunicação de resultados e de documentação de IA responsável: figuras exploratórias, de avaliação e de interpretabilidade, além do **Model Card** e do **Datasheet for Datasets**. Um modelo sem documentação de uso pretendido, limitações e desempenho por subgrupo não deveria ser publicado — muito menos num domínio em que um erro tem consequência clínica.

Entradas / Saídas

- **Classe**: `ReportingStage` em `src/pipelines/reporting.py`.
- **Entrada obrigatória**: `paths.data.user_features`. As figuras de avaliação são tratadas como opcionais — a etapa tolera a ausência de artefatos de etapas anteriores.

- **Entradas opcionais** (lidas conforme disponibilidade): `data.user_features` (distribuição de classes, perfil de atividade, projeção de embeddings), `data.tweets_labeled` + `data.user_labels` (frequência de palavras, n-grams, nuvem de palavras, evolução de sentimento, atividade circadiana, rede de similaridade), `reports/metrics/evaluation.json` (métricas, matrizes de confusão, calibração, fatias), `reports/tables/model_comparison.csv`, `reports/metrics/predictions.csv` (curvas ROC/PR), `reports/interpretability/*.csv`, `reports/ablation/ablation_summary.csv`, `reports/metrics/labeling_quality.json` e `features_summary.json` (usados pelos templates de Model Card/Datasheet).
- **Saídas:** figuras em `reports/figures/*.png` e `*.svg`, `reports/model_cards/model_card.md`, `reports/datasheets/datasheet.md`.

Implementação (passo a passo)

1. `apply_theme(figure_dpi)` (`src/visualization/theme.py`) aplica o tema visual global, garantindo paleta e formatação consistentes em todas as figuras da execução.
2. **Figuras exploratórias** (`_exploratory_figures`): a partir de `user_features` — `distribuicao_classes`, `perfil_atividade`, `projecao_embeddings` (UMAP→t-SNE→PCA em cascata conforme disponibilidade). Se houver `tweets_labeled`: `frequencia_palavras`, `bigramas`, `nuvem_palavras` (None se `wordcloud` não instalado), `evolucao_sentimento`, `atividade_circadiana` (com a janela de insônia sombreada), `mapa_atividade` (heatmap dia×hora) e `rede_similaridade` (grafo de similaridade lexical via TF-IDF + cosseno + `networkx` — substituto de rede de interação real, já que menções são removidas na anonimização LGPD).
3. **Figuras de avaliação** (`_evaluation_figures`): a partir de `evaluation.json` (ausência = warning + skip, não falha) — `comparacao_modelos` (barras horizontais com IC 95%); por modelo, `matriz_confusao_<modelo>` (normalizada por linha, para não deixar a classe majoritária dominar visualmente), `calibracao_<modelo>` (curva de confiabilidade com ECE), `fatias_<modelo>` (desempenho por subgrupo comportamental); `curvas_roc` e `curvas_precisao_revocacao` para o melhor modelo (via `predictions.csv`).
4. **Figuras de interpretabilidade** (`_interpretability_figures`): varre `reports/interpretability/` por padrões de arquivo, gerando `importancia_<modelo>`, `importancia_grupo_<modelo>`, `shap_<modelo>` (coloridas por grupo de atributo); se existir `ablation_summary.csv`, gera `ablacao` (contribuição marginal leave-one-out + desempenho isolado por grupo).
5. Cada figura passa por `_save()`, que tolera `None` (dependência ausente/dados insuficientes, logado em debug) e delega a `visualization.theme.save_figure()` (formatos e DPI configurados, sempre fechando a figura para não esgotar memória).
6. **Documentos de IA responsável** (se `generate_model_card: true`):
 - `build_model_card()` (`reports_templates/model_card.py`), estrutura de Mitchell et al. (2019): detalhes do modelo, uso pretendido/fora de escopo, dados de treinamento (com qualidade de rotulação e kappa de Cohen), métricas agregadas + IC, desempenho por classe (destaque para a revocação de "Ideação Suicida"), calibração, desempenho por subgrupo comportamental (com nota sobre a impossibilidade de auditoria demográfica por minimização de dados LGPD), limitações, considerações éticas e recomendações de uso;
 - `build_datasheet()` (`reports_templates/datasheet.py`), estrutura de Gebru et al. (2021): motivação, composição, processo de coleta, pré-processamento/rotulação, privacidade e base legal LGPD, limitações/vieses conhecidos, distribuição (explicitamente **não** redistribuído) e manutenção.

7. `run()` consolida as figuras geradas, loga sucesso/tentativas, registra artefatos no tracker de experimentos (se configurado) e retorna o resumo.

Configuração relevante (`configs/evaluation.yaml`, bloco `reporting`)

```
reporting:
  formats: [json, csv, md]
  figure_formats: [png, svg]
  figure_dpi: 300
  save_predictions: true
  generate_model_card: true
```

também consome `evaluation.metrics.primary` (escolha do melhor modelo), `evaluation.uncertainty.confidence_level` (IC exibido no Model Card) e `features.temporal.insomnia_window` (janela sombreada na figura de atividade circadiana). Caminhos de saída vêm de `ReportPaths` (`src/config/paths.py`).

Design notável

- Tolerância a dependências ausentes: funções de figura retornam `None` (em vez de lançar exceção) quando falta um pacote opcional (`wordcloud`, `umap-learn`, `networkx`) ou dado insuficiente, permitindo que a etapa complete mesmo com etapas anteriores parcialmente executadas.
- Tema visual único e ordinal (`CLASS_COLORS` segue a ordem de severidade `controle` → `depressao` → `ideacao_suicida`, distinguível em escala de cinza e sob deuteranopia/protanopia).
- PNG + SVG por padrão (inserção rápida vs. diagramação vetorial final).
- Rede de similaridade (TF-IDF/cosseno) em vez de rede de interação real — decisão explícita de privacidade, já que menções são removidas na anonimização LGPD.
- Ausência de auditoria demográfica é declarada como limitação, não contornada: o projeto não coleta atributos sensíveis por minimização de dados, e o Model Card documenta essa lacuna como decisão consciente.